Projet Othello - L3 Informatique

Julia, Andréa, Lucile, Sarah

Mai 2026

Introduction

- **Jeu d'Othello en langage C**
- Deux modes de jeu :
  - Joueur VS Joueur
  - Joueur VS IA
- Objectifs :
  - gestion du plateau
  - règles
  - IA

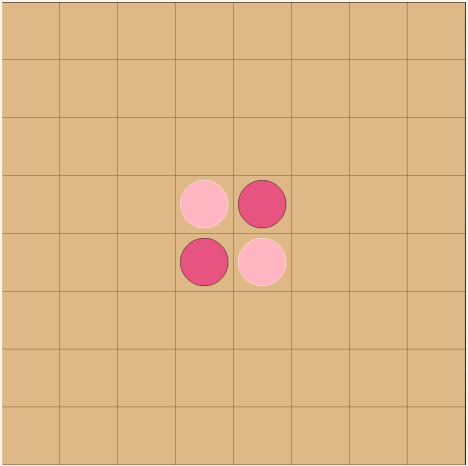
Introduction

1. Introduction, règles du jeu et interface d'accueil
2. Mode Joueur VS Joueur et complexité
3. Mode Joueur VS IA, fonction d'évaluation, Minimax et complexité
4. Alpha-Beta, fonctionnalités et sauvegarde

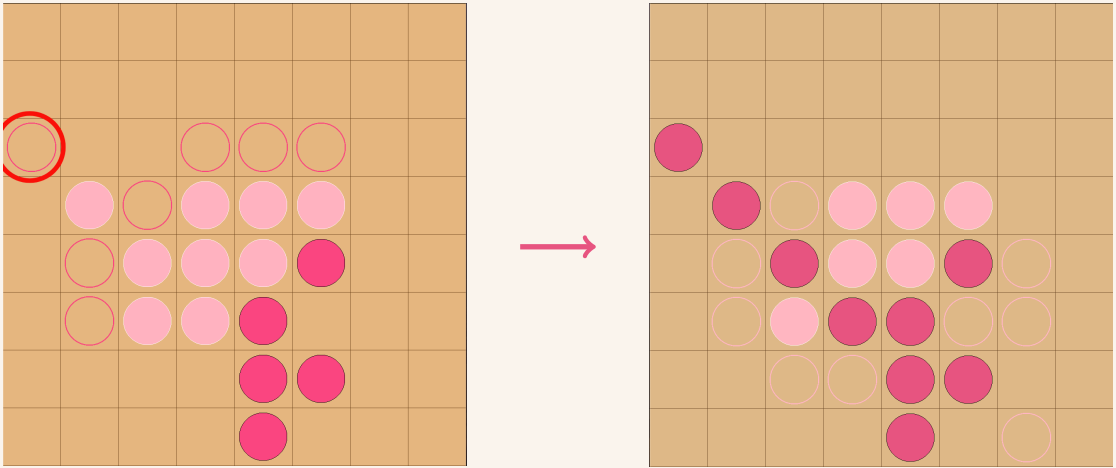
Règles du jeu - Plateau de départ

Joueur 1

Joueur 2

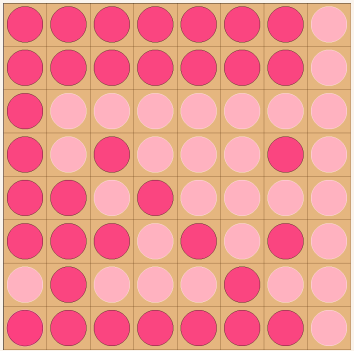


Règles du jeu - Capture des pions



Règles du jeu - Fin de partie

- Plateau complet
- Un joueur n'a plus de pions
- Aucun des deux joueurs ne peut jouer



exemple d'un plateau complet

Interface d’accueil

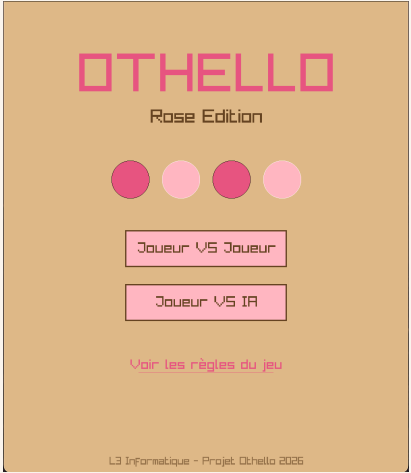


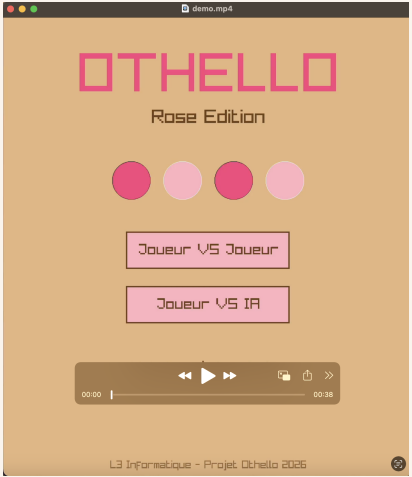
Photo de l’interface d’accueil

Photo du pop-up avec règles du jeu

Interface d’accueil - Saisie des noms

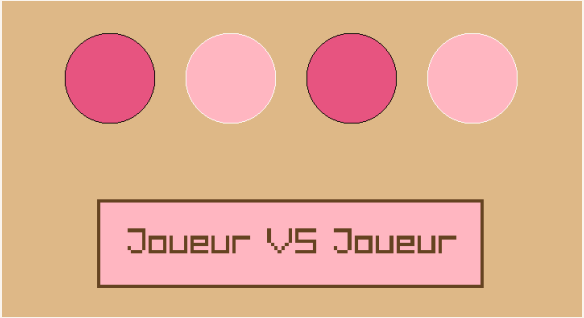
- Saisie du nom de chaque joueur
- Personnalisation

Démonstration

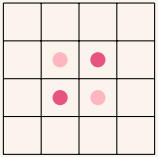


Mode joueur contre joueur

- Deux joueurs humains
- Base du projet
- Nécessaire pour l'IA



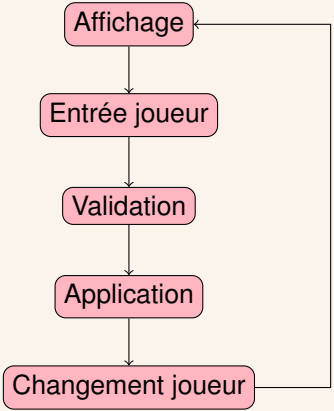
Plateau 8×8



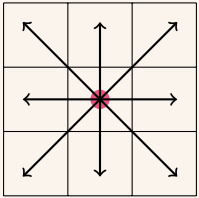
Initialisation

- Matrice 2D
- 3 états (vide / rose foncé / rose clair)

Boucle du jeu



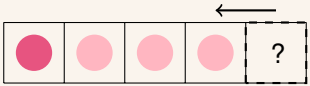
Validation : 8 directions



Scan directions

- 8 directions testées
- Recherche “sandwich”

Validation d'un coup (exemple)

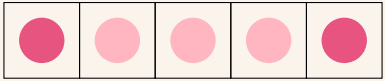


On encadre les pions adverses

- Case voisine adverse obligatoire
- On avance dans la direction
- On doit retrouver notre pion

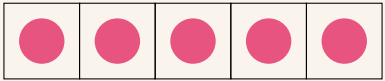
Principe de capture

Avant



Encadrement

Après



Changement de couleur

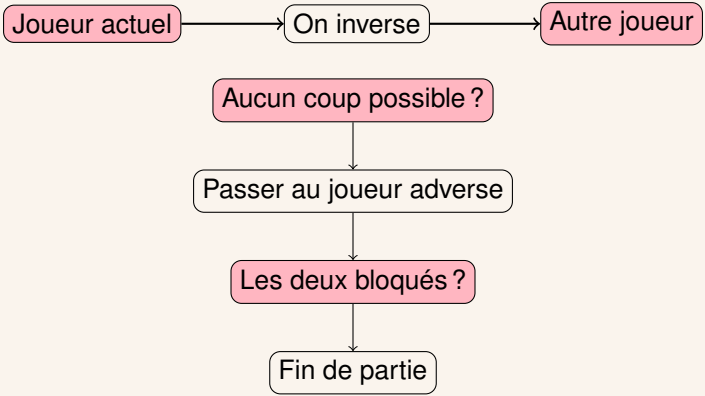
Mise à jour du score

Compteur rose foncé

Compteur rose clair

- Double boucle sur la matrice 8×8
- Comptage des pions
- Complexité constante

Changement de joueur / Fin de partie



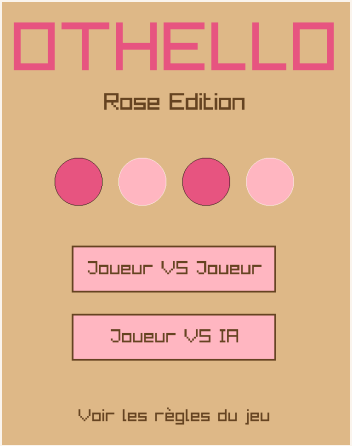
- Comptage final des pions
- Comparaison des scores
- Détermination du gagnant

Complexité

- Taille fixe du plateau
- Parcours toujours limité
- Complexité en pratique : $O(1)$

Conclusion Mode joueur contre joueur

Joueur vs Joueur → **Fiabilité du moteur de jeu** → Joueur vs IA

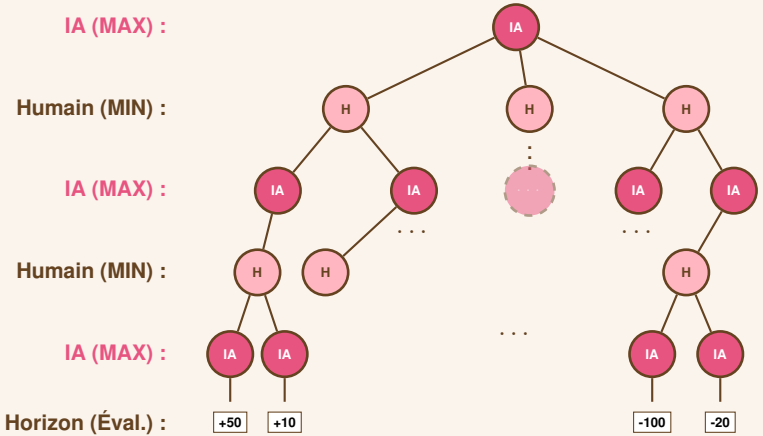


L'Intelligence Artificielle

Mode : Joueur contre IA

L'Algorithme Minimax

Simulation de l'arbre de décision



Fonction d'évaluation

- **Principe :** Attribuer une valeur fixe à chaque case du plateau.
- **Points Forts (+100) :** Les coins sont définitifs et servent d'ancres.
- **Points Faibles (-50) :** Les cases adjacentes aux coins (cases X) risquent de "donner" le coin à l'adversaire.

100	-20	10	5	5	10	-20	100
-20	-50	-2	-2	-2	-2	-50	-20
10	-2	5	1	1	5	-2	10
5	-2	1	0	0	1	-2	5
5	-2	1	0	0	1	-2	5
10	-2	5	1	1	5	-2	10
-20	-50	-2	-2	-2	-2	-50	-20
100	-20	10	5	5	10	-20	100

Hiérarchie des cases stratégiques

Fonction d'évaluation

- **Définition :** C'est le nombre de coups valides qu'un joueur peut jouer à son tour.

Calcul du score

$$Score = MesCoupsPossibles - CoupsAdversaire$$

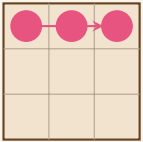
(Différence des mobilités)

- **L'objectif :** « Étouffer » l'adversaire pour le forcer à jouer sur les cases à malus (adjacentes aux coins).

Fonction d'évaluation

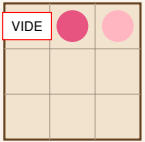
- **Définition :** Un pion est dit « stable » s'il est mathématiquement impossible pour l'adversaire de le retourner d'ici la fin de la partie.
- **La règle :** La stabilité se propage à partir des coins vers le long des bords.

Cas Stable (Ancré)



Propagation depuis le coin

Cas Instable (Vulnérable)



Risque de capture (sandwich)

Fonction d'évaluation

En début de partie :

- **Piège :** Trop de pions créent des « cibles » faciles pour l'adversaire.
- **Risque :** Perdre en mobilité.
- **IA :** Donne un poids **négligeable** au score (Poids 1).

En fin de partie :

- **Changement :** L'IA prend en compte le nombre de pions restant dès qu'il reste moins de 10 cases vides.
- **IA :** Le nombre de pions devient un critère **important** (Poids 100).

Phase de jeu	Coefficient	Stratégie de l'IA
Début / Milieu	**1**	Privilégier le placement et la mobilité
Fin (10 cases vides)	**100**	**Priorité au nombre de pions**

La Prise de Décision

- **Lister et Trier :** Identifier les coups légaux et privilégier les cases fortes (coins).
- **Simuler :** Tester chaque option sur une copie virtuelle du plateau.
- **Anticiper :** Utiliser le **Minimax** pour prédire le score 5 coups plus tard.
- **Sélectionner :** Retenir le coup qui garantit la meilleure note finale.

Analyse de la complexité

- **Complexité Minimax :** $O(b^d)$

Les paramètres

- b : nombre moyen de coups possibles (≈ 10).
- d : profondeur de recherche (5).

Croissance exponentielle

- Pour $d = 5$: **100 000** positions.
- Pour $d = 6$: **1 000 000** positions.
- Pour $d = 10$: **10 milliards**.

Constat : Plus on veut une IA intelligente (d élevé), plus le temps de calcul explose.

Défi : L'explosion combinatoire

Comment augmenter les performances de notre ia sans ralentir le jeu ?

$\rightarrow$ **L'élagage Alpha-Bêta**